PROJECT DOCUMENTATION

AI FAQ ASSISTANT

Retrieval-Augmented Generation (RAG) Based FAQ Question Answering System

PROJECT TEAM

S.No.	Name	Register Number
1	PRABAKARAN .S	C24UG206CCS015
2	SALMAN .M	C24UG206CCS018
3	SURYA NARAYANA .S	C24UG206CCS020
4	VIKNESH T.U	C24UG206CCS022

1. ABSTRACT

The AI FAQ Assistant is a Retrieval-Augmented Generation (RAG) based application designed to answer user questions from trusted Frequently Asked Questions (FAQ) or help-page content. The system collects relevant website content using a web scraper, removes unnecessary HTML elements, divides the extracted text into smaller overlapping chunks, and converts the chunks into numerical embeddings. These embeddings are stored in a local FAISS vector index for fast semantic retrieval. When a user submits a question, the application searches the knowledge base for the most relevant passages and provides those passages as context to a Claude language model. The model then generates a concise answer using only the retrieved context instead of relying on unsupported information. This architecture reduces the risk of hallucinated answers and allows the knowledge base to be updated by re-scraping the source content. The project is implemented using Python, Flask, BeautifulSoup, Sentence Transformers, FAISS, and the Anthropic API.

Keywords: RAG, FAQ Assistant, Semantic Search, Embeddings, FAISS, Flask, Claude API, Web Scraping

2. INTRODUCTION

Traditional FAQ systems generally depend on exact keyword matching or manually prepared responses. Such systems may fail when a user asks a question using different words from the original FAQ. The proposed AI FAQ Assistant addresses this limitation through semantic retrieval. Instead of matching only exact words, the system represents text and user questions as embeddings and identifies passages with similar meaning.

3. SYSTEM OBJECTIVES

• To automatically collect FAQ/help information from permitted web pages.

• To clean and divide website content into manageable text passages.

• To create semantic embeddings for the FAQ knowledge base.

• To retrieve the most relevant passages for each user question using FAISS.

• To generate concise, context-grounded answers through a large language model.

• To return source information along with the generated answer for better traceability.

4. SYSTEM ARCHITECTURE

Stage	Component	Function
1	**FAQ / Help Website**	Provides the source content.
2	**Web Scraper**	Requests pages and extracts readable text using BeautifulSoup.
3	**Chunker**	Splits page text into 180-word chunks with 30-word overlap.
4	**Embedding Model**	all-MiniLM-L6-v2 converts text into 384-dimensional vectors.
5	**FAISS Vector Index**	Stores vectors and performs similarity search.
6	**Flask API**	Receives questions through the /ask endpoint.
7	**Claude LLM**	Uses retrieved context to generate the final answer.

5. MAIN MODULES

scraper.py – Fetches permitted web pages, removes navigation/script/style noise, extracts the main readable content, and saves it as scraped_pages.json.

chunker.py – Splits extracted text into overlapping passages so that relevant information can be retrieved without processing an entire page at once.

embed_store.py – Generates embeddings, normalizes vectors, builds a FAISS IndexFlatIP index, and provides semantic search for user queries.

app.py – Runs the Flask API. The /health endpoint checks availability, while /ask validates the question, retrieves relevant chunks, calls Claude, and returns the answer with source details.

6. WORKING PROCEDURE

Step 1 – Content Collection: The user provides one or more permitted FAQ/help URLs. The scraper downloads the HTML content and extracts the useful text while removing scripts, navigation, headers, footers and other noise.

Step 2 – Text Chunking: The extracted text is divided into passages of approximately 180 words with a 30-word overlap. The overlap helps preserve context between adjacent passages.

Step 3 – Embedding and Indexing: Each passage is converted into a vector using the all-MiniLM-L6-v2 Sentence Transformer model. The vectors are normalized and stored in FAISS for similarity-based retrieval.

Step 4 – Question Retrieval: A user's question is converted into the same embedding space. FAISS searches for the top four relevant passages and assigns similarity scores.

Step 5 – Grounded Answer Generation: Relevant passages above the configured similarity threshold are supplied to Claude together with the question. The prompt instructs the model to answer only from the provided context and avoid guessing.

Step 6 – API Response: The Flask API returns a JSON response containing the generated answer and the source title, URL and similarity score.

7. TECHNOLOGIES USED

Technology	Purpose
Python	Core implementation language
Flask	REST API and application server
Requests + BeautifulSoup	Web-page retrieval and HTML parsing
Sentence Transformers	Semantic text embeddings
FAISS	Local vector similarity search
Anthropic Claude API	Context-grounded natural-language answer generation
JSON / .env	Data storage and configuration

8. EXPECTED OUTPUT

For a valid question, the system returns an answer generated from the retrieved FAQ context together with the relevant source details. If no sufficiently relevant information is found, the system responds that the information is not available in the FAQ knowledge base instead of generating an unsupported answer. The /health endpoint provides a simple status response to confirm that the Flask service is running.

9. FUTURE ENHANCEMENTS

The system can be extended with a web-based chat interface, conversation memory, scheduled content refresh, user feedback collection, authentication, automated evaluation metrics, and a hosted vector database such as ChromaDB, Pinecone or Weaviate. These enhancements can improve usability, maintainability and deployment readiness.

10. CONCLUSION

The AI FAQ Assistant demonstrates how RAG can combine semantic retrieval with a language model to produce answers grounded in a controlled knowledge source. The modular design separates data collection, chunking, indexing, retrieval and answer generation, making the system suitable for academic demonstration and further development.